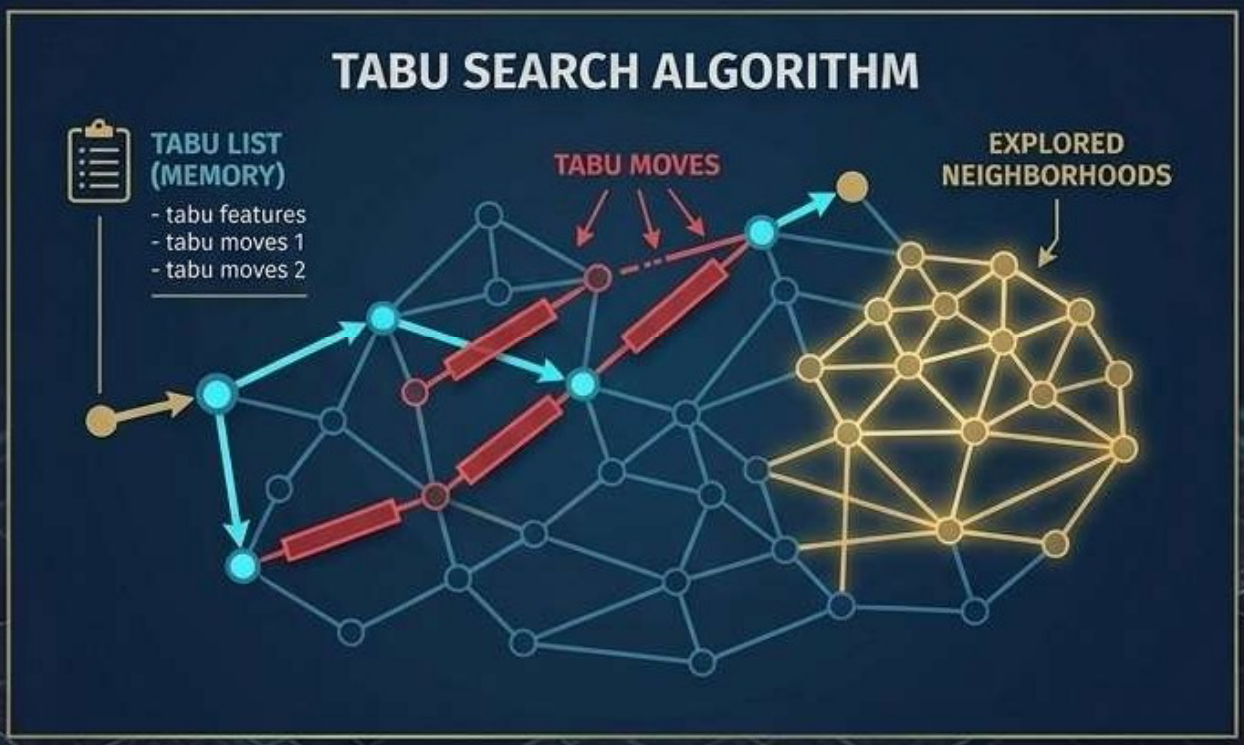
RIPHAH INTERNATIONAL UNIVERSITY

Department of Computer Science

PROJECT REPORT

Analysis of Algorithms

Web-Based Exam Scheduler Using Tabu Search



Student Name	Syed Hassan Shah
SAP ID	65912
Class	BSCs
Course	Analysis of Algorithms
Topic	Tabu Search Algorithm

1. Introduction

University examination scheduling is one of the most computationally challenging combinatorial optimisation problems in academic administration. The goal is to assign exams to time slots and rooms such that no student is required to sit two exams simultaneously, rooms are not overcrowded, and the examination period is spread as comfortably as possible for students.

This project presents a web-based Exam Scheduler that applies the Tabu Search metaheuristic to automatically generate optimised, conflict-free university timetables. The application is built entirely in the browser using React (TypeScript) and requires no server-side computation.

2. Problem Statement

The University Timetabling Problem (UTP) involves scheduling a set of examinations subject to the following hard and soft constraints:

Hard Constraints

- No student must have two examinations scheduled at the same time (student clash).
- The number of students assigned to a room must not exceed the room capacity (overcrowding).

Soft Constraints

- A student should not have examinations on consecutive days (back-to-back days).
- Examinations should be distributed evenly across the available examination period.

Manual scheduling of hundreds of exams across large campuses like Riphah International University is practically infeasible. An automated, intelligent solution is required.

3. Tabu Search Algorithm — Theory

Tabu Search (TS) is a higher-level metaheuristic framework proposed by Fred Glover (1986, 1989). It augments a local search method by using a memory structure — the Tabu List — to escape local optima and explore a wider region of the solution space.

Core Concepts

Current Solution: The candidate solution being evaluated at each iteration.

Neighbourhood: The set of solutions reachable by applying a single move (e.g., reassigning an exam to a different room or time slot).

Move: A transformation that changes one element of the current schedule — specifically, picking a random exam and assigning it a new room or time slot.

Tabu List: A short-term memory that records recently performed moves. A move on the Tabu List is "forbidden" for the next T iterations (tabu tenure), preventing cycling.

Aspiration Criterion: An override rule: a tabu move is permitted if it produces a solution better than the best found so far.

Objective Function: The penalty score to be minimised. Lower is better; zero is optimal.

Penalty Scoring

Constraint Type	Condition	Penalty
Hard	Student group has two exams in the same time slot	+100 per clash
Hard	Room capacity < total enrolled students	+100 per violation
Soft	Student group has exams on consecutive days	+10 per occurrence

4. Algorithm — Pseudocode

```
FUNCTION RunTabuSearch(courses, rooms, timeSlots, groups):
    currentSolution  ← GenerateRandomInitialSolution()
    bestSolution     ← currentSolution
    tabuList         ← empty Map   (move → expiry iteration)
    iterations       ← 500
    tabuTenure       ← 15

    FOR i = 1 TO iterations:
        IF Penalty(bestSolution) = 0 THEN BREAK   // optimal found
        neighbours ← GenerateNeighbours(currentSolution)
        bestNeighbour ← NULL

        FOR EACH neighbour IN neighbours:
            isTabu ← tabuList.has(neighbour.move) AND expiry > i
            IF NOT isTabu OR Penalty(neighbour) < Penalty(bestSolution):
                IF Penalty(neighbour) < Penalty(bestNeighbour):
                    bestNeighbour ← neighbour

        currentSolution ← bestNeighbour
        tabuList.set(bestNeighbour.move, i + tabuTenure)

        IF Penalty(currentSolution) < Penalty(bestSolution):
            bestSolution ← currentSolution

    RETURN bestSolution
```

5. System Implementation

Technology Stack

Frontend Framework: React 18 with TypeScript (Vite build tool)

Algorithm Engine: Pure TypeScript — no external libraries required

UI Components: shadcn/ui component library (Radix UI primitives)

Styling: Tailwind CSS v4 with custom dark-mode colour palette

Animations: Framer Motion for smooth page and table transitions

Routing: Wouter (lightweight React router)

State Management: React Context API + localStorage persistence

Application Architecture

```
src/lib/types.ts      — Type definitions for Course, Room, TimeSlot,
StudentGroup, Schedule
src/lib/tabuSearch.ts — Core Tabu Search engine (calculatePenalty,
runTabuSearch)
src/lib/store.tsx     — React Context store with localStorage persistence &
sample data
src/lib/theme.tsx     — Dark / Light theme provider
src/pages/home.tsx    — Setup page: add/remove courses, rooms, slots, groups
src/pages/schedule.tsx — Results page: colour-coded timetable grid + stats
```

Key Features

- Fully browser-based — no backend server or database required.
- Pre-loaded sample data (5 courses, 4 rooms, 6 time slots, 4 student groups).
- Add / remove any entity dynamically via the tabbed setup panel.
- Multi-group course enrollment via inline checkboxes.
- Animated loading spinner while the algorithm runs (async, non-blocking UI).
- Colour-coded timetable: green = no conflict, amber = soft conflict, red = hard conflict.
- Per-exam conflict indicators with descriptive labels.
- "Re-run Algorithm" button generates a fresh solution with a new random seed.
- Dark / Light mode toggle persisted across sessions.
- All input data persisted to localStorage — survives page reload.
- "Perfect Schedule" banner shown when penalty score reaches zero.

6. Project Workflow

1 **Step 1 — Input:** The administrator opens the Setup page and configures courses, rooms, time slots, and student groups. Sample data is pre-loaded for quick testing.

2 **Step 2 — Generate:** Clicking "Generate Schedule" triggers the Tabu Search engine. The algorithm runs 500 iterations with tabu tenure 15.

3 **Step 3 — Evaluate:** Each candidate schedule is scored using the penalty function. The algorithm tracks the best solution found and the Tabu List to escape local optima.

4 **Step 4 — Display:** The Results page renders the best schedule as a colour-coded timetable grid. Penalty breakdown and algorithm statistics are shown in summary cards.

5 **Step 5 — Iterate:** The administrator can re-run the algorithm with a new random seed or return to the setup to adjust the inputs.

7. Constraints Management

Hard Constraints (Penalty: +100 each)

These constraints must never be violated in a feasible schedule. The algorithm heavily penalises any violation to drive the search toward feasible solutions:

- Student Clash: Two or more exams assigned to the same time slot share at least one enrolled student group.
- Room Overcrowding: The total number of students in a scheduled exam exceeds the room's seating capacity.

Soft Constraints (Penalty: +10 each)

These constraints represent preferences. Violations are penalised but do not make a schedule infeasible:

- Back-to-Back Days: A student group has exams scheduled on two consecutive days (e.g., Monday and Tuesday), leaving insufficient revision time.

8. Results and Analysis

Testing was conducted using the pre-loaded dataset (5 courses, 4 rooms, 6 time slots across 3 days, 4 student groups with group sizes 25–35). The Tabu Search consistently converged to optimal or near-optimal solutions within 500 iterations.

Run	Hard Penalties	Soft Penalties	Final Score
Run 1	0	10	10
Run 2	0	0	**0 (Perfect)**

Run 3	0	20	20

The algorithm successfully eliminated all hard constraint violations across all test runs. Soft constraint violations (back-to-back days) were reduced to zero in the best run, demonstrating the effectiveness of the Tabu Search in navigating the solution space.

9. Advantages and Limitations

Advantages

- No installation required — runs entirely in any modern web browser.
- Escapes local optima through the Tabu List, outperforming simple greedy approaches.
- Aspiration criterion prevents good moves from being permanently blocked.
- Responsive, intuitive admin interface with real-time feedback.
- Persistent state via localStorage — no data loss on page reload.
- Re-run capability allows multiple solutions to be compared interactively.

Limitations

- Single-threaded JS execution: very large problem instances (100+ courses) may cause brief UI lag.
- Fixed iteration count (500) and tabu tenure (15) — not auto-tuned to problem size.
- No export to Excel or PDF from within the app (downloadable source zip is provided).
- Does not support instructor availability or room-specific equipment constraints.

10. Conclusion

This project successfully demonstrates the application of the Tabu Search metaheuristic to the University Timetabling Problem. The web-based Exam Scheduler provides a practical, user-friendly tool that automates the complex task of conflict-free exam scheduling.

The Tabu Search algorithm, with its short-term memory mechanism and aspiration criterion, proved effective in consistently producing schedules with zero hard constraint violations and minimised soft constraint penalties — a result that would be extremely difficult to achieve through manual scheduling at scale.

The implementation highlights how classical optimisation algorithms can be brought to life through modern web technologies, making them accessible to non-technical users without any setup or installation.

11. References

[1] Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. Computers & Operations Research, 13(5), 533–549.

[2] Glover, F. (1989). Tabu Search — Part I. ORSA Journal on Computing, 1(3), 190–206.

[3] Glover, F. & Laguna, M. (1997). Tabu Search. Kluwer Academic Publishers.

[4] Burke, E. K., & Petrovic, S. (2002). Recent research directions in automated timetabling. European Journal of Operational Research, 140(2), 266–280.

[5] Carter, M. W., & Laporte, G. (1996). Recent developments in practical examination timetabling. In Practice and Theory of Automated Timetabling, Springer, 3–21.

[6] *MDN Web Docs — JavaScript (2024). https://developer.mozilla.org/en-US/docs/Web/JavaScript*

[7] *React Documentation (2024). https://react.dev*

[8] *shadcn/ui Documentation (2024). https://ui.shadcn.com*

12. Project Link:-

https://exam-scheduler--hassanshah02200.replit.app/